LAB SESSION NO. 1

PROLOG-FACTS

OBJECTIVE:

To learn how to define and implement facts.

THEORY:

A **fact** is a predicate expression that makes a declarative statement about the problem domain. Whenever a variable occurs in a Prolog expression, it is assumed to be **universally quantified**. Note that all Prolog sentences must end with a terminator.

Facts consist of:

- A predicate name (or functor) such as likes, lectures, office, etc. This must begin with a lower case letter.
- Zero or more arguments, such as alison, ai3, and s134.

Note that facts (and rules, and questions) must end with a full stop.

Syntax for Facts

- Names of relationships & objects must be **atoms**.
- The relationship is written first then the objects
- The **full stop** character, '.' **must come at the end of a fact**

There are following two types of Facts.

1 Simple facts

2 Facts with Arguments

SIMPLE FACTS:

In Prolog, we can make some statements by using facts. Facts either consist of a particular item or a relation between items. For example we can represent the fact that it is sunny by writing the program:

sunny.

We can now ask a query of Prolog by asking

?- sunny.

?- is the Prolog prompt. To this query, Prolog will answer **yes**. sunny is true because (from above) **Prolog matches it in its database of facts if it is save in the facts.**

Facts have some simple rules of **syntax**. Facts should always begin with a **lowercase letter** and end with a **full stop**. The facts themselves can consist of any letter or number combination, as well as the **underscore _** character. However, names containing the characters **-,+,*,/,** or other mathematical operators should be **avoided**.

FACTS WITH ARGUMENTS:

More complicated facts consist of a relation and the items that this refers to. These items are called arguments. Facts can have arbitrary number of arguments from zero upwards. A general model is shown below:

relation(<argument1>,<argument2>,....,<argumentN>).

The arguments can be any legal Prolog term. The basic Prolog terms are an integer, an atom, a variable or a structure. Various Prolog implementation enhance this basic list with other data types, such as floating point numbers, or strings.

Examples of simple Facts:

Here are some simple facts about an imaginary world. /* and */ are comment delimiters

```
john_has_cold.              /* john has cold */
it_is_raining.              /* it is raining */
john_Forgot_His_Raincoat.   /*  john forgot his raincoat */
fred_lost_his_car_keys.     /* fred lost his car keys */
peter_footballer.           /* peter plays football */
```

These describe a particular set of circumstances for some character john. We can interrogate this database of facts, by again posing a query. For example:

```
?- john_Forgot_His_Raincoat.
yes

?- it_is_raining.
yes

?- foggy.
no
```

The first two queries succeed since they can be matched against facts in the database above. However, foggy fails (since it cannot be matches) and Prolog answers no since we have not told it this fact.

Exercise: Given the database below, study the queries below it. Indicate whether you think the goal will succeed or not by answering yes or no as prompted.

```
blue_box.
red_box.
green_circle.
blue_circle.
orange_triangle.

?- green_circle.
?- circle_green.
?- red_triangle.
?- red_box.
?- orange_Triangle.
```

Facts with Arguments Examples 1

An example knowledgebase. It details who eats what in some world model.

```
eats(fred,oranges).          /* "Fred eats oranges" */
eats(fred,t_bone_steaks).    /* "Fred eats T-bone steaks" */
eats(tony,apple).            /* "Tony eats apple" */
eats(john,apple).            /* "John eats apple" */
eats(john,grapefruit).       /* "John eats grapefruit" */
```

Exercise: If ask some queries then we would get the following interaction:

```
?- eats(fred,oranges).      /* does this match anything in the database? */

yes                         /* yes, matches the first clause in the database */

?- eats(john,apple).   /*  do we have a fact that says john eats apples? */

yes                         /* yes we do, clause 4 of our eats database */

?- eats(mike,apple).        /* how about this query, does mike eat apples */

no                          /* not according to the above database. */

?- eats(fred,apple).        /* does fred eat apples */

no                          /* again no, we don't know whether fred eats apples */
```

Facts with Arguments Examples 2

Let us consider another knowledgebase. This time we will use the predicate age to indicate the ages of various individuals.

```
age(john,32).          /*  John is 32 years old */

age(agnes,41).         /*  Agnes is 41 years old*/

age(george,72).        /*  George is 72 years old*/

age(ian,2).            /*  Ian is 2 years old*/

age(thomas,25).        /*  Thomas is 25 years old*/
```

Exercise: If we now ask some queries we would get the following interaction:

```
?- age(ian,2).         /* is Ian 2 years old? */

yes                    /* yes, matches against the fourth clause of age */

?- agnes(41).          /* for some relation agnes are they 41  */

no         /* No.   In the database above we only know about the relation */

           /*  age, not about the relation agnes, so the query fails */

?- age(ian,two)        /* is Ian  two years old? */

no         /* No.  two and 2 are not the same and therefore don't match */
```

Example:

likes(ali,mary).

In the above fact john and mary are two atoms. Atoms are usually made from letters and digits with lowercase characters. The underscore (_) can also be used to separate 2 words but is not allowed as the first character. Atoms can also be legally made from symbols. The followings are legal atoms.

hello

zz42

The followings are **not** legal atoms:

Hello

4hello

_Hello

The fact “likes(john,mary).” say that there is a relation between john and mary. It can be read as either john likes mary or mary likes john. This reversibility can be very useful to the programmer, however it can also be a source of mistakes. You have to be clear on how you intend to interpret the relation.

Variables and Unification

How do we say something like "What does Fred eat"? Suppose we had the following fact in our knowledgebase:

eats(fred,mangoes).

How do we ask what fred eats. We could type in something like

?- eats(fred,what).

However Prolog will say no. The reason for this is that what does not match with mangoes. In order to match arguments in this way we must use a Variable. The process of matching items with variables is known as unification. Variables are distinguished by starting with a capital letter. Here are some examples:

X /* a capital letter */

VaRiAbLe /* a word - it be made up or either case of letters */

My_name /* we can link words together via '_' (underscore) */

Thus returning to our first question we can find out what fred eats by typing

?- eats(fred,What).

What=mangoes

yes

As a result of this query, the variable What has matched (or unified) with mangoes. We say that the variable “What” now has the binding mangoes. When we pose a query, if the query is successful, Prolog prints both the variable and the variable name, as we see above.

Variable Examples 1

Let's consider some examples using facts. First consider the following database.

```
loves(john,mary).

loves(fred,hobbies).
```

Now let's look at some simple queries using variables

```
?- loves(john,Who).      /* Who does john love? */

Who=mary                 /* yes , Who gets bound to mary */

yes                      /*  and the query succeeds*/

?- loves(arnold,Who).    /* does arnold love anybody */

no                       /* no,  arnold doesn't match john or fred */

?- loves(fred,Who).      /* Who does fred love */

Who = hobbies            /*  Note the to Prolog Who is just the name of a variable, it */

yes                      /* semantic connotations are not picked up, hence  Who unifies */

                         /* with hobbies */
```

Variable Examples 2

Here are some more difficult object/variable comparisons. Consider the following knowledgebase showing a library of cassette tapes. Notice the final argument to tape, which is the name of a favourite song from the album.

```
tape(1,van_morrison,astral_weeks,madam_george).

tape(2,beatles,sgt_pepper,a_day_in_the_life).

tape(3,beatles,abbey_road,something).

tape(4,rolling_stones,sticky_fingers,brown_sugar).

tape(5,eagles,hotel_california,new_kid_in_town).
```

Let us now look at some queries.

?- tape(5,Artist,Album,Favorite_Song). /* what are the contents of tape 5 */

Artist=eagles

Album=hotel_california

Favorite_Song=new_kid_in_town

yes

?- tape(4,rolling_stones,sticky_fingers,Song). /* find just song */

Song=brown_sugar /* which you like best from the album */

yes

Exercise 3 - Variable Exercise

Here are some problems for which unification sometimes succeeds and sometimes fails. The items to be compared are labelled with Variables. If you think the unification succeeds, write down any bindings made e.g. if (say) X gets bound to foo, then write X=foo as the substitution.

eats(fred,tomatoes)

eats(Whom,What) /* do these unify (match) */

Above will be unify?

eats(fred,Food)

eats(Person,jim) /* do these unify (match) */

Above will be unify?

cd(29,beatles,sgt_pepper).

cd(A,B,help). /* do these unify (match) */

Above will be unify?

f(X,a)

f(a,X) /* do these unify (match) */

Above will be unify?

likes(jane,X)

likes(X,jim) /* do these unify (match) */

Above will be unify?

f(X,Y)

f(P,P) /* do these unify (match) */

Above will be unify?

f(foo,L)

f(A1,A1) /* do these unify (match) */

Above will be unify?

Check Your Answers

HOW TO QUERY?

Once you have entered the facts in a program then you can ask prolog about it. An example program can be:

eats(ali,oranges). /* 'Ali eats oranges' */

eats(ahmed,apple). /* 'Ahmed eats apple' */

eats(ali,apple). /* 'Ali eats apple' */

If we ask now some queries then we would get the following output (answers):

?- eats(ali,oranges).

/* does this match anything in the knowledgebase? */

yes

/* yes, that match the first clause */

?- eats(ali,apple).

yes

?- eats(khan,apple).

no

/* there is no relation between khan and apple */

Now if we ask:

?- eats(ali,What).

The first answer will be "What=apple" because that is the unification that match the first relation of eats with ali in the facts. Then prolog will be waiting for you to press a key. If you press enter Prolog will be ready for a new query. In most implementation if you press the key (**semicolon**) **''**;**''** then Prolog will try to find if there is any other successful unification. Prolog will give the second result "What=orange" because this is the second one in the program. If you press again the key **''**;**''** then Prolog will try to find a third unification. The result will be **''no''** because compiler is not able to find any other successful unification.

?- eats(ali,What).

What=apple ;

What=oranges ;

no

Consider this example of program that could be used by a library:

book(1,title1,author1).

book(2,title2,author1).

book(3,title3,author2).

book(4,title4,author3).

Now if we want to know if we have a book from the author2 we can ask:

?- book(_,_,author2).

yes

If we want to know which book from the author1 we have:

?- book(_,X,author1).

X=title1 ;

X=title2 ;

Example for Practice:

lectures(alia, calculus).

lectures(ahmed, databases).

female(alia).

age(alia, 29).

office(alia, r14).

animal(lion).

animal(sparrow).

has_feathers(sparrow).

Try any query by yourself first and check the answers.

Lab EXERCISE:

1. How do you add the following facts?

Mark likes anything in hunger.
Mark likes any type of cars.
Mark likes anyone who likes him.
Dany is tall.
Napoleon is short.
Lemons are sour.
Susan is the sister of John.
Charlie Parker plays piano.
A piano is a musical instrument.
Shaquille plays basketball with friends.
Sam is a smart guy.

After adding the above facts you have to write some different queries to find the answers related to Facts.

2. has(jack,apples).

has(ann,plums).

has(dan,money).
fruit(apples).
fruit(plums).

From the clauses/facts given above, answer the following. Convert the given questions in query using predicate. Should depict the given questions into form of query predicates.

- **What has Jack?**
- **Does Jack have something?**
- **Who has apples and who has plums?**
- **Does someone have apples and plums?**
- **Has Dan fruits?**